element from an enumerable then the method is subtract, which is different from the arithmetic methods we'll see in a bit because it changes the set in-place. The clear method empties the set entirely.

For conditional deletion, sets define delete_if, which takes a block and deletes items for which the block returns a truthy value, and the inverse keep_if, which takes a block and deletes items for which the block returns a falsey value. (Sets also define select!, which is equivalent to keep_if, and reject!, which is equivalent to delete_if, except that the bang methods return nil if the set doesn't change.)

Comparing Sets

Set comparison is a little bit different than comparisons of other enumerable types. Two sets are eql? or == if they contain the same elements and order doesn't matter. A set is === to an object if the object is in the set, making sets useful in case statements. The === operator is aliased as both member?, and include?. Sets also define length as the number elements in the set, and empty? as having more than 0 elements.

Sets implement Comparable and the <=> operator, but the implementation is not quite what you may be expecting. The <=> mimics the behavior of eql?, meaning that it returns 0 if the two set have the same elements. The greater then/less than behavior is based only on the relationship between the the two sets, not the values of the elements. If the left set is a proper subset of the right set, <=> returns -1. If the right set is a proper subset of the left set, <=> returns 1. If the two sets do not have a subset/superset relationship, meaning that each has elements not in the other set, than <=> returns nil, which is very unusual given the way <=> works in most o the rest of Ruby.

So < is aliased as proper_subset? and <= is aliased as subset?, > is proper_superset? and >= is superset?. Sets also define disjoint?, which is true if the sets have no common element and intersect?, which is true if the sets have at least one common element. For example:

```
a = Set[1, 2, 3]
b = Set[3, 2, 1]
c = Set[2, 1]
d = Set[1, 2, 3, 4]
e = Set[2, 3, 4, 5]

a == b               # => true
b === 3              # => true
b === "3"            # => false
c <=> b              # => -1
d <=> c              # => 1
d < c                # => false
d > c                # => true
d <=> e              # => nil
d.disjoint?(e)       # => false
d.intersect?(e)      # => true
```

Set Operations

Sets respond to arithmetic operators similar to the way arrays do:

- Set union returns a new set containing all elements from either operand, with no duplicates. The method is union, it's aliased as + and | .